RuntimeException
BaralhoVazioException
CartaNaoEncontradaException
JogoInvalidoException
MaoCheiaException
com
map
games
blackjack
MainBlackjack
+ main(args : String[])
usa
BlackjackRegras
+ ACAO_HIT : AcaoJogador
+ ACAO_STAND : AcaoJogador
+ criarBaralho() : BaralhoTradicional
+ cartasIniciaisPorJogador() : int
+ acoesPossiveis(jogador, partida) : List<AcaoJogador>
+ executarAcao(jogador, acao, partida)
+ partidaFinalizada(partida) : boolean
+ getVencedor(partida) : Jogador
usa
BlackjackUtils
+ calcularValorMao(mao) : int
+ isBlackjack(mao) : boolean
+ estourou(mao) : boolean
supertrunfo
CartaSuperTrunfo
- atributos : Map<AtributoSuperTrunfo, Integer>
- isSuperTrunfo : boolean
+ exibir() : String
+ getAtributo(atributo) : int
+ isSuperTrunfo() : boolean
tem
1..*
AtributoSuperTrunfo
POTENCIA
VELOCIDADE
ACELERACAO
PRECO
CILINDRADAS
MainSuperTrunfo
+ main(args : String[])
usa
SuperTrunfoRegras
+ criarBaralho() : BaralhoSuperTrunfo
+ cartasIniciaisPorJogador() : int
+ acoesPossiveis(jogador, partida) : List<AcaoJogador>
+ executarAcao(jogador, acao, partida)
+ partidaFinalizada(partida) : boolean
+ getVencedor(partida) : Jogador
cria
BaralhoSuperTrunfo
+ criarCartas() : List<Carta>
usa
usa
framework
RegrasJogo
B extends Baralho
+ criarBaralho() : B
+ cartasIniciaisPorJogador() : int
+ maximoJogadores() : int
+ minimoJogadores() : int
+ acoesPossiveis(jogador, partida) : List<AcaoJogador>
+ executarAcao(jogador, acao, partida)
+ partidaFinalizada(partida) : boolean
+ getVencedor(partida) : Jogador
+ aposDistribuicaoInicial(partida)
+ antesDaRodada(partida)
+ aposRodada(partida)
AcaoJogador
- nome : String
- descricao : String
GameFacade
+ criarPartida(jogadores, regras) : Partida
+ executarPartida(partida) : Jogador
+ executarPartida(partida, usarConsoleLog) : Jogador
cria/executa
instancia
instancia
Partida
B extends Baralho, R extends RegrasJogo<B>
# jogadores : List<Jogador>
# baralho : B
# regras : R
# listeners : List<PartidaListener>
# rodadaAtual : int
+ adicionarListener(listener)
+ removerListener(listener)
+ notificar(evento)
+ {final} jogarPartida() : Jogador
+ getJogadores() : List<Jogador>
+ getBaralho() : B
+ getRegras() : R
+ getRodadaAtual() : int
# inicializarPartida()
# distribuirCartasIniciais()
# rodarTurnos()
# executarTurno(jogador)
# finalizarPartida() : Jogador
# hookAposDistribuicao()
# hookAntesDaRodada()
# hookAposRodada()
PartidaEvento
- tipo : TipoEvento
- mensagem : String
- jogador : Jogador
- carta : Carta
+ getTipo() : TipoEvento
+ getMensagem() : String
+ getJogador() : Jogador
+ getCarta() : Carta
PartidaSimples
B extends Baralho, R extends RegrasJogo<B>
notifica
1..*
PartidaListener
+ onEvento(evento : PartidaEvento)
possui
1..*
Jogador
- nome : String
- pontuacao : int
- estrategia : EstrategiaJogador
+ getNome() : String
+ getPontuacao() : int
+ adicionarPontos(pontos)
+ setEstrategia(estrategia)
+ decidirAcao(partida, acoes) : AcaoJogador
+ escolherCarta(opcoes, partida) : Optional<Carta>
referencia
0..1
tem
1
TipoEvento
PARTIDA_INICIADA
RODADA_FINALIZADA
CARTA_DISTRIBUIDA
CARTA_JOGADA
TURNO_INICIADO
TURNO_FINALIZADO
JOGADOR_ELIMINADO
PARTIDA_FINALIZADA
MENSAGEM
usa
1
B
usa
1
R
Baralho
# cartas : List<Carta>
- embaralhador : EmbaralhadorStrategy
+ embaralhar()
+ comprarCarta() : Carta
+ isVazio() : boolean
+ tamanho() : int
+ getCartasNaoModificaveis() : List<Carta>
+ resetar()
+ setEmbaralhador(embaralhador)
+ criarCartas() : List<Carta>
PartidaConsoleListener
+ onEvento(evento : PartidaEvento)
possui
1
Mao
- cartas : List<Carta>
- limiteMaximo : int
+ adicionarCarta(carta)
+ removerCarta(carta) : boolean
+ removerCarta(indice) : Carta
+ tamanho() : int
+ isVazia() : boolean
+ isCheia() : boolean
+ getCartas() : List<Carta>
+ limpar()
referencia
delega
cria
1
EstrategiaJogador
+ decidirAcao(jogador, partida, acoes) : AcaoJogador
+ escolherCarta(jogador, opcoes, partida) : Optional<Carta>
usa
1
EmbaralhadorStrategy
+ embaralhar(cartas : List<Carta>)
contém
1..*
contém
0..*
0..1
Carta
# identificador : String
# viradaParaCima : boolean
+ getIdentificador() : String
+ isViradaParaCima() : boolean
+ virarParaCima()
+ virarParaBaixo()
+ exibir() : String
BaralhoTradicional
- qtdBaralhos : int
+ criarCartas() : List<Carta>
EstrategiaIAAleatoria
+ decidirAcao(jogador, partida, acoes) : AcaoJogador
+ escolherCarta(jogador, opcoes, partida) : Optional<Carta>
EstrategiaHumanaConsole
+ decidirAcao(jogador, partida, acoes) : AcaoJogador
+ escolherCarta(jogador, opcoes, partida) : Optional<Carta>
EmbaralhadorAleatorio
+ embaralhar(cartas : List<Carta>)
SemEmbaralhamento
+ embaralhar(cartas : List<Carta>)
CartaTradicional
- valor : int
- naipe : Naipe
+ exibir() : String
tem
1
Naipe
OUROS
COPAS
ESPADAS
PAUS